```
%pip install tensorflow numpy matplotlib
```

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
```

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train[y_train == 0]
x_test = x_test[np.isin(y_test, [0, 1])]

y_test = y_test[np.isin(y_test, [0, 1])]

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)
print("Test labels:", np.unique(y_test))
```

```
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)
print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

```
input_layer = keras.Input(shape=(784,))
# Encoder
encoded = layers.Dense(128, activation="relu")(input_layer)
encoded = layers.Dense(32, activation="relu")(encoded)
# Decoder
decoded = layers.Dense(128, activation="relu")(encoded)
output_layer = layers.Dense(784, activation="sigmoid")(decoded)
autoencoder = keras.Model(input_layer, output_layer)
autoencoder.summary()
```

```
autoencoder.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)
```

```
history = autoencoder.fit(
    x_train,
    x_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1
)
```

```
reconstructed = autoencoder.predict(x_test)
error = np.mean((x_test - reconstructed) ** 2, axis=1)
threshold = np.percentile(error[y_test == 0], 95)
print("Anomaly threshold:", threshold)
```

```
predictions = error > threshold
for i in range(10):
    if predictions[i]:
        print("Image", i, "- Anomaly")
    else:
        print("Image", i, "- Normal")
```

```
plt.figure(figsize=(10, 4))
for i in range(5):
    # Original image
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title("Original")
    plt.axis("off")
    # Reconstructed image
    plt.subplot(2, 5, i + 6)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")
plt.tight_layout()
plt.show()
```